

Git Quiz



Git Quiz

Version Control Basics



The wheel picks the student — **one student, one question!**

wheelofnames.com

What does `git add file.txt` do?

1 Deletes the file from disk

2 Commits the file immediately

3 Uploads the file to the remote

4 Moves the file into the staging area, ready to be committed



Question 1

Git

What does `git add file.txt` do?

1 Deletes the file from disk

2 Commits the file immediately

3 Uploads the file to the remote

4 Moves the file into the staging area, ready to be committed



✓ Answer 4 — `git add` moves changes into the staging area (the index) — it does not commit them yet



What does `git commit -m "message"` **do?**

- 1 Uploads staged changes to the remote repository
- 2 Saves a permanent snapshot of the staged changes to the local history
- 3 Discards all staged changes
- 4 Renames the current branch



What does `git commit -m "message"` **do?**

1 Uploads staged changes to the remote repository

2 Saves a permanent snapshot of the staged changes to the local history ✓

3 Discards all staged changes

4 Renames the current branch

✓ Answer 2 — `git commit` takes whatever is staged and records it as a new snapshot in the local history



What's the difference between `git diff` and `git diff --staged` ?

1 They do exactly the same thing

2 `git diff` shows staged changes, `git diff --staged` shows unstaged changes

3 `git diff` shows unstaged changes, `git diff --staged` shows what will go into the next commit

4 `git diff --staged` only works after pushing



What's the difference between `git diff` and `git diff --staged` ?

1 They do exactly the same thing

2 `git diff` shows staged changes, `git diff --staged` shows unstaged changes

3 `git diff` shows unstaged changes, `git diff --staged` shows what will go into the next commit



4 `git diff --staged` only works for pushing



What is `.gitignore` for?

- 1 Tells Git which files/patterns to never track (e.g. `node_modules`, `.env`)
- 2 Lists files that must always be committed
- 3 Encrypts sensitive files
- 4 Is required for git init to work



What is `.gitignore` for?

1 Tells Git which files/patterns to never track (e.g. `node_modules`, `.env`) ✓

2 Lists files that must always be committed

3 Encrypts sensitive files

4 Is required for git init to work

✓ Answer 1 — `.gitignore` lists patterns for files Git should never track — build output, secrets, dependencies, etc.



What does `git checkout -b` feature do?

1 Deletes the current branch

2 Only works if the branch already exists

3 Merges the new branch into main automatically

4 Creates a new branch AND switches to it in one step



What does `git checkout -b` feature do?

1 Deletes the current branch

2

Only works if the branch already exists

3

Merges the new branch into main automatically

4

Creates a new branch AND switches to it in one step



✓ Answer 4 — `git checkout -b` (or the newer git switch `-c`) creates the branch and checks it out immediately



When does a **merge conflict** happen?

- 1 Git always picks the newer change automatically
- 2 Two branches changed the same lines differently — Git needs you to resolve it manually
- 3 The remote repository is down
- 4 You forgot to run git init



When does a **merge conflict** happen?

1 Git always picks the newer change automatically

2 Two branches changed the same lines differently — Git needs you to resolve it manually



3 The remote repository is down

4 You forgot to run git init

✓ Answer 2 — A merge conflict happens when the same lines were changed differently on both sides — Git can't guess which one you want



What does HEAD refer to in Git?

- 1 A pointer to the commit your working directory is currently on
- 2 The very first commit ever made
- 3 The name of the default branch
- 4 A hidden backup of your repository



What does HEAD refer to in Git?

1 A pointer to the commit your working directory is currently on ✓

2 The very first commit ever made

3 The name of the default branch

4 A hidden backup of your repository

✓ Answer 1 — HEAD is simply a moving pointer to whatever commit (usually via a branch) you currently have checked out



What does `git stash` do?

- 1 Permanently deletes your uncommitted changes
- 2 Temporarily shelves uncommitted changes so you can switch branches with a clean working directory
- 3 Commits your changes with a placeholder message
- 4 Uploads uncommitted changes to the remote



What does `git stash` do?

1 Permanently deletes your uncommitted changes

2 Temporarily shelves uncommitted changes so you can switch branches with a clean working directory ✓

3 Commits your changes with a placeholder message

4 Uploads uncommitted changes to the remote

✓ Answer 2 — `git stash` saves uncommitted changes on a temporary shelf — `git stash pop` brings them back later



What's the difference between `git fetch` and `git pull`?

1 They are identical commands

2 `git fetch` downloads remote commits without merging them; `git pull` fetches AND merges into your current branch

3 `git fetch` merges automatically; `git pull` only downloads

4 `git pull` only works the very first time you clone a repo



What's the difference between `git fetch` and `git pull`?

1 They are identical commands

2 `git fetch` downloads remote commits without merging them; `git pull` fetches AND merges into your current branch



3 `git fetch` merges automatically; `git pull` only downloads

4 `git pull` only works the very first time you clone a repo



What does `git remote add origin <url>` **do?**

- 1 Downloads the repository from GitHub
- 2 Registers the name "origin" as a shortcut for a remote repository's URL
- 3 Immediately pushes all your commits to GitHub
- 4 Deletes your local repository and replaces it with the remote one



What does `git remote add origin <url>` do?

1 Downloads the repository from GitHub

2 Registers the name "origin" as a shortcut for a remote repository's URL 

3 Immediately pushes all your commits to GitHub

4 Deletes your local repository and replaces it with the remote one

 **Answer 2** — `git remote add origin` just registers the alias "origin" for that URL — it doesn't push or pull anything by itself





Well Done!

That's the end of the Git quiz.

Add, commit, branch, merge, HEAD, stash, fetch/pull, remotes: ✓